

AUC-ROC Curve in Machine Learning Clearly Explained



50+ Exciting Industry Projects to become a Full-Stack Data Scientist 🧑💻

[Download Projects](#) ✕

[Home](#)

[aniruddha](#) — June 16, 2020

[Beginner](#) [Python](#) [Technique](#)

AUC-ROC Curve – The Star Performer!

You’ve built your machine learning model – so what’s next? You need to evaluate it and validate how good (or bad) it is, so you can then decide on whether to implement it. That’s where the AUC-ROC curve comes in.

The name might be a mouthful, but it is just saying that we are calculating the “Area Under the Curve” (AUC) of “Receiver Characteristic Operator” (ROC). Confused? I feel you! I have been in your shoes. But don’t worry, we will see what these terms mean in detail and everything will be a piece of cake!



imgflip.com

For now, just know that the AUC-ROC curve helps us visualize how well our machine learning classifier is performing. Although it works for only binary classification problems, we will see towards the end how we can extend it to evaluate multi-class classification problems too.

We’ll cover topics like sensitivity and specificity as well since these are key topics behind the AUC-ROC curve.

I suggest going through the article on [Confusion Matrix](#) as it will introduce some important terms which we will be using in this article.

Table of Contents

- What are Sensitivity and Specificity?
- Probability of Predictions

We use cookies on Analytics Vidhya websites to deliver our services, analyze web traffic, and improve your experience on the site. By using Analytics Vidhya, you agree to our [Privacy Policy](#) and [Terms of Use](#). [Accept](#)

AUC-ROC Curve in Machine Learning Clearly Explained



What are Sensitivity and Specificity?

This is what a confusion matrix looks like:

		ACTUAL VALUES	
		POSITIVE	NEGATIVE
PREDICTED VALUES	POSITIVE	TP	FP
	NEGATIVE	FN	TN

From the confusion matrix, we can derive some important metrics that were not discussed in the previous article. Let's talk about them here.

Sensitivity / True Positive Rate / Recall

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

Sensitivity tells us what proportion of the positive class got correctly classified.

A simple example would be to determine what proportion of the actual sick people were correctly detected by the model.

False Negative Rate

$$FNR = \frac{FN}{TP + FN}$$

False Negative Rate (FNR) tells us what proportion of the positive class got incorrectly classified by the classifier.

A higher TPR and a lower FNR is desirable since we want to correctly classify the positive class.

Specificity / True Negative Rate

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Specificity tells us what proportion of the negative class got correctly classified.

Taking the same example as in Sensitivity, Specificity would mean determining the proportion of healthy people who were correctly identified by the model.

False Positive Rate

$$FPR = \frac{FP}{TN + FP} = 1 - \text{Specificity}$$

AUC-ROC Curve in Machine Learning Clearly Explained



build an evaluation metric. But before that, let’s understand why the probability of prediction is better than predicting the target class directly.

Probability of Predictions

A machine learning classification model can be used to predict the actual class of the data point directly or predict its probability of belonging to different classes. The latter gives us more control over the result. We can determine our own threshold to interpret the result of the classifier. This is sometimes more prudent than just building a completely new model!

Setting different thresholds for classifying positive class for data points will inadvertently change the Sensitivity and Specificity of the model. And one of these thresholds will probably give a better result than the others, depending on whether we are aiming to lower the number of False Negatives or False Positives.

Have a look at the table below:

ID	Actual	Prediction Probability	>0.6	>0.7	> 0.8	Metric
1	0	0.98	1	1	1	
2	1	0.67	1	0	0	
3	1	0.58	0	0	0	
4	0	0.78	1	1	0	
5	1	0.85	1	1	1	
6	0	0.86	1	1	1	
7	0	0.79	1	1	0	
8	0	0.89	1	1	1	
9	1	0.82	1	1	1	
10	0	0.86	1	1	1	
			0.75	0.5	0.5	TPR
			1	1	0.66	FPR
			0	0	0.33	TNR
			0.25	0.5	0.5	FNR

The metrics change with the changing threshold values. We can generate different confusion matrices and compare the various metrics that we discussed in the previous section. But that would not be a prudent thing to do. Instead, what we can do is generate a plot between some of these metrics so that we can easily visualize which threshold is giving us a better result.

The AUC-ROC curve solves just that problem!

What is the AUC-ROC curve?

The **Receiver Operator Characteristic (ROC)** curve is an evaluation metric for binary classification problems. It is a probability curve that plots the **TPR** against **FPR** at various threshold values and essentially **separates the ‘signal’ from the ‘noise’**. The **Area Under the Curve (AUC)** is the measure of the ability of a classifier to distinguish between classes and is used as a summary of the ROC curve.

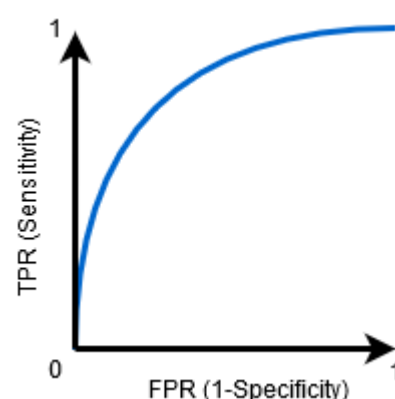
The higher the AUC, the better the performance of the model at distinguishing between the positive and negative classes.



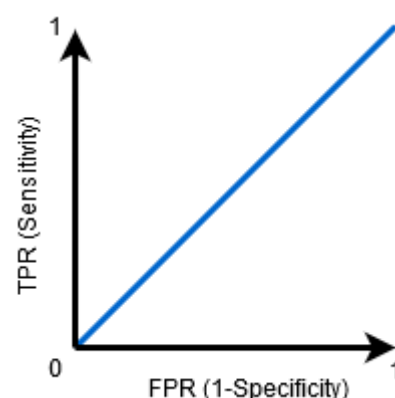


AUC-ROC Curve in Machine Learning Clearly Explained

correctly. If, however, the AUC had been 0, then the classifier would be predicting all Negatives as Positives, and all Positives as Negatives.



When $0.5 < \text{AUC} < 1$, there is a high chance that the classifier will be able to distinguish the positive class values from the negative class values. This is so because the classifier is able to detect more numbers of True positives and True negatives than False negatives and False positives.



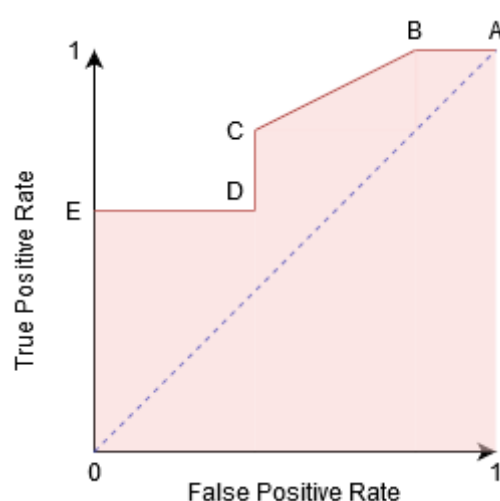
When $\text{AUC} = 0.5$, then the classifier is not able to distinguish between Positive and Negative class points. Meaning either the classifier is predicting random class or constant class for all the data points.

So, the higher the AUC value for a classifier, the better its ability to distinguish between positive and negative classes.

How Does the AUC-ROC Curve Work?

In a ROC curve, a higher X-axis value indicates a higher number of False positives than True negatives. While a higher Y-axis value indicates a higher number of True positives than False negatives. So, the choice of the threshold depends on the ability to balance between False positives and False negatives.

Let's dig a bit deeper and understand how our ROC curve would look like for different threshold values and how the specificity and sensitivity would vary.



We can try and understand this graph by generating a confusion matrix for each point corresponding to a threshold and talk about the performance of our classifier:

We use cookies on Analytics Vidhya websites to deliver our services, analyze web traffic, and improve your experience on the site. By using Analytics Vidhya, you agree to our [Privacy Policy](#) and [Terms of Use](#). [Accept](#)

AUC-ROC Curve in Machine Learning Clearly Explained



Predicted	+ve	5	5
	-ve	0	0

Point A is where the Sensitivity is the highest and Specificity the lowest. This means all the Positive class points are classified correctly and all the Negative class points are classified incorrectly.

In fact, any point on the blue line corresponds to a situation where True Positive Rate is equal to False Positive Rate.

All points above this line correspond to the situation where the proportion of correctly classified points belonging to the Positive class is greater than the proportion of incorrectly classified points belonging to the Negative class.

		Actual	
		+ve	-ve
Predicted	+ve	5	4
	-ve	0	1

Although Point B has the same Sensitivity as Point A, it has a higher Specificity. Meaning the number of incorrectly Negative class points is lower compared to the previous threshold. This indicates that this threshold is better than the previous one.

Point C			Point D		
Actual			Actual		
	+ve	-ve		+ve	-ve
Predicted +ve	4	2	Predicted +ve	3	2
Predicted -ve	1	3	Predicted -ve	2	3
TPR/Sensitivity = 0.8			TPR/Sensitivity = 0.6		
FPR = 0.4			FPR = 0.4		
Specificity = 1-FPR = 0.6			Specificity = 1-FPR = 0.6		

Between points C and D, the Sensitivity at point C is higher than point D for the same Specificity. This means, for the same number of incorrectly classified Negative class points, the classifier predicted a higher number of Positive class points. Therefore, the threshold at point C is better than point D.

Now, depending on how many incorrectly classified points we want to tolerate for our classifier, we would choose between point B or C for predicting whether you can defeat me in PUBG or not.

"False hopes are more dangerous than fears."-J.R.R. Tolkein

Point E		
Actual		
	+ve	-ve
Predicted +ve	3	0
Predicted -ve	2	5
TPR/Sensitivity = 0.6		
FPR = 0		
Specificity = 1-FPR = 1		

Point E is where the Specificity becomes highest. Meaning there are no False Positives classified by the model. The model can correctly classify all the Negative class points! We would choose this point if our problem was to give perfect song recommendations to our users.



AUC-ROC Curve in Machine Learning Clearly Explained

that both, the Sensitivity and Specificity, would be the highest and the classifier would correctly classify all the Positive and Negative class points.

Understanding the AUC-ROC Curve in Python

Now, either we can manually test the Sensitivity and Specificity for every threshold or let [sklearn](#) do the job for us. We're definitely going with the latter!

Let's create our arbitrary data using the sklearn make_classification method:

```
1  from sklearn.datasets import make_classification
2  from sklearn.model_selection import train_test_split
3
4  # generate two class dataset
5  X, y = make_classification(n_samples=1000, n_classes=2, n_features=20, random_state=27)
6
7  # split into train-test sets
8  X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=27)
```

AUC-ROC1.py hosted with ❤ by GitHub [view raw](#)

I will test the performance of two classifiers on this dataset:

```
1  # train models
2  from sklearn.linear_model import LogisticRegression
3  from sklearn.neighbors import KNeighborsClassifier
4
5  # logistic regression
6  model1 = LogisticRegression()
7  # knn
8  model2 = KNeighborsClassifier(n_neighbors=4)
9
10 # fit model
11 model1.fit(X_train, y_train)
12 model2.fit(X_train, y_train)
13
14 # predict probabilities
15 pred_prob1 = model1.predict_proba(X_test)
16 pred_prob2 = model2.predict_proba(X_test)
```

AUC-ROC2.py hosted with ❤ by GitHub [view raw](#)

Sklearn has a very potent method roc_curve() which computes the ROC for your classifier in a matter of seconds! It returns the FPR, TPR, and threshold values:

```
1  from sklearn.metrics import roc_curve
2
3  # roc curve for models
4  fpr1, tpr1, thresh1 = roc_curve(y_test, pred_prob1[:,1], pos_label=1)
5  fpr2, tpr2, thresh2 = roc_curve(y_test, pred_prob2[:,1], pos_label=1)
6
7  # roc curve for tpr = fpr
8  random_probs = [0 for i in range(len(y_test))]
9  p_fpr, p_tpr, _ = roc_curve(y_test, random_probs, pos_label=1)
```

AUC-ROC3.py hosted with ❤ by GitHub [view raw](#)

The AUC score can be computed using the roc_auc_score() method of sklearn.



AUC-ROC Curve in Machine Learning Clearly Explained

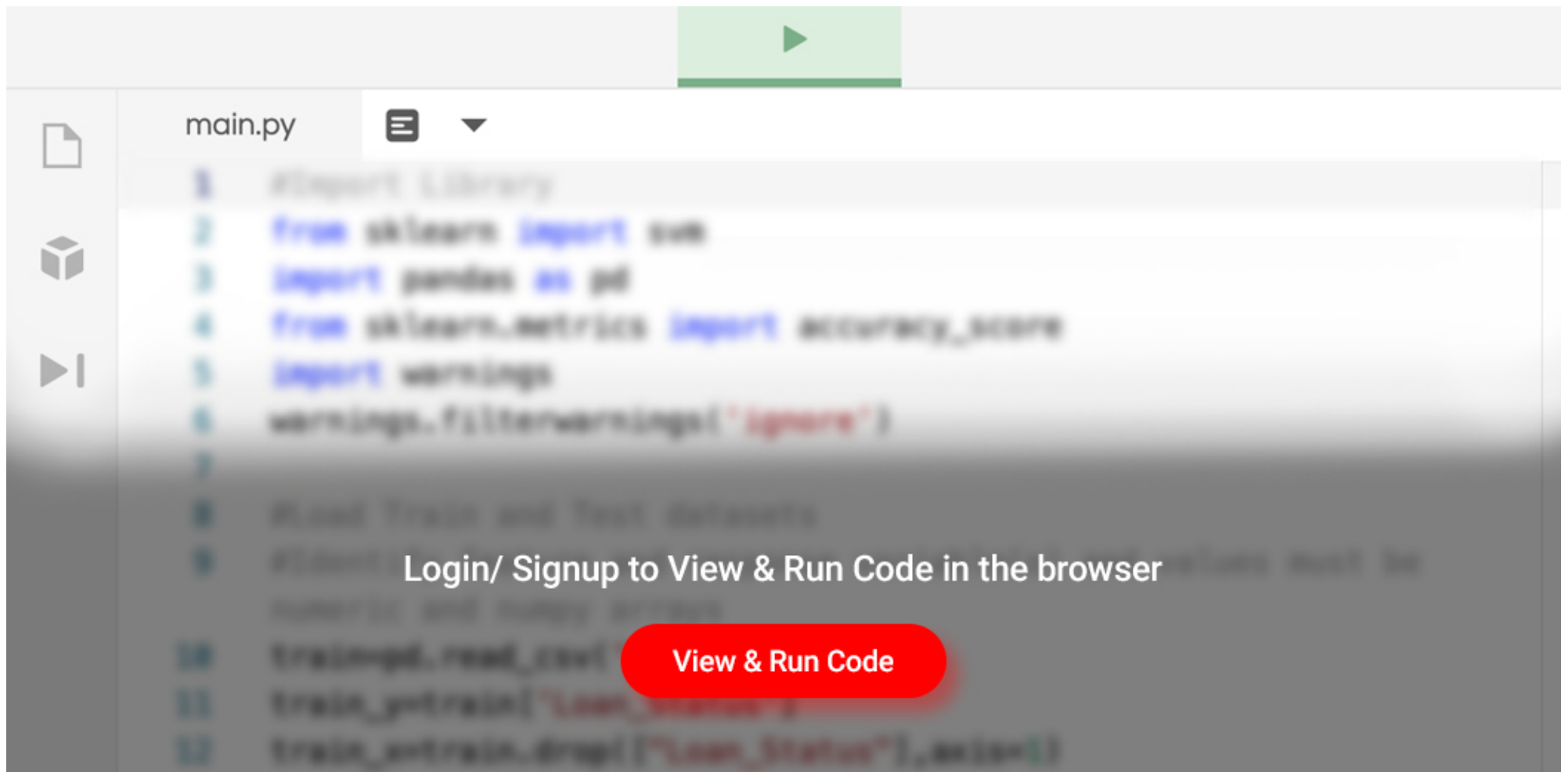
```
4 auc_score1 = roc_auc_score(y_test, pred_prob1[:,1])
5 auc_score2 = roc_auc_score(y_test, pred_prob2[:,1])
6
7 print(auc_score1, auc_score2)
```

AUC-ROC4.py hosted with ❤ by GitHub

[view raw](#)

0.9761029411764707 0.9233769727403157

Try this code out in the live coding window below:



We can also plot the ROC curves for the two algorithms using [matplotlib](#):

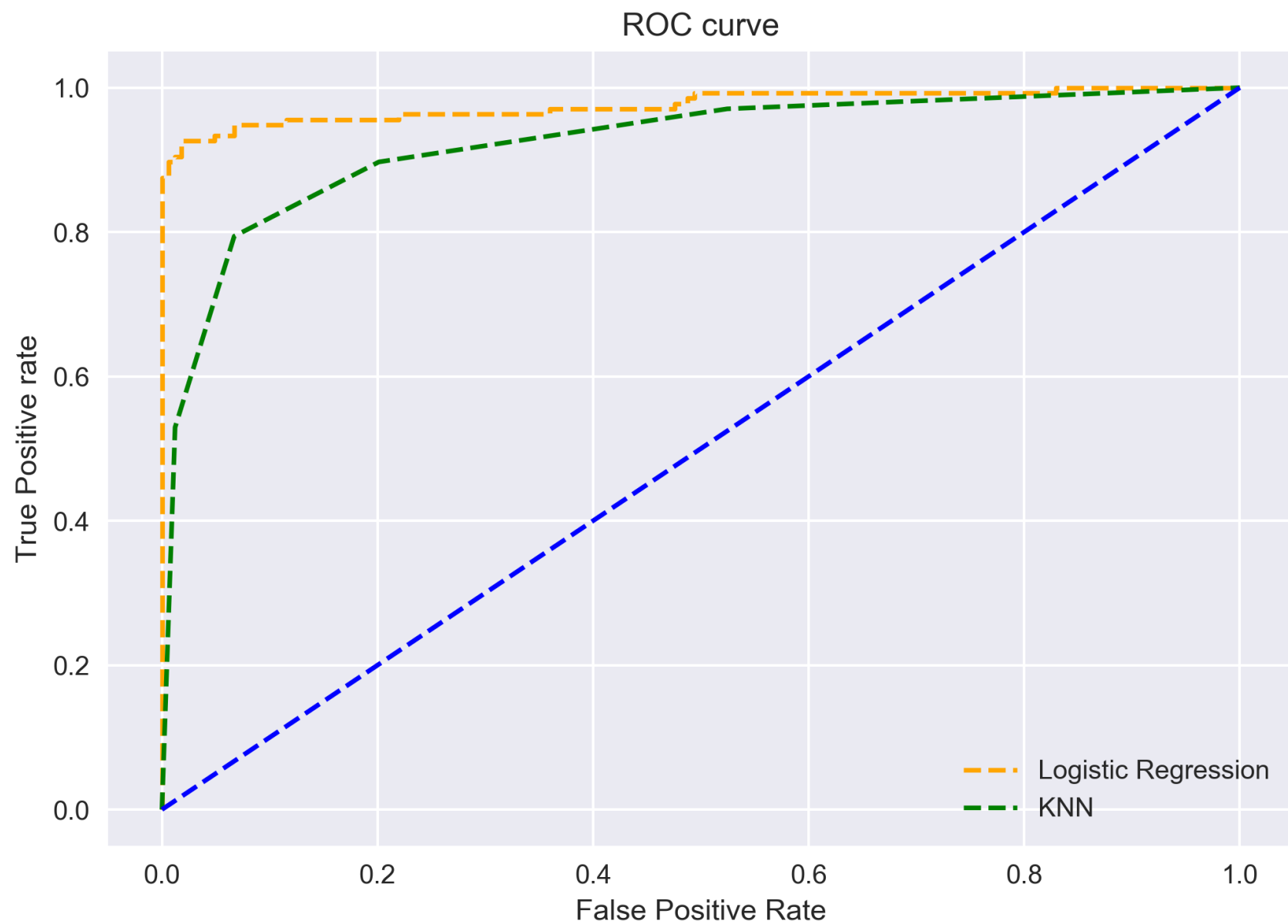
```
1 # matplotlib
2 import matplotlib.pyplot as plt
3 plt.style.use('seaborn')
4
5 # plot roc curves
6 plt.plot(fpr1, tpr1, linestyle='--',color='orange', label='Logistic Regression')
7 plt.plot(fpr2, tpr2, linestyle='--',color='green', label='KNN')
8 plt.plot(p_fpr, p_tpr, linestyle='--', color='blue')
9 # title
10 plt.title('ROC curve')
11 # x label
12 plt.xlabel('False Positive Rate')
13 # y label
14 plt.ylabel('True Positive rate')
15
16 plt.legend(loc='best')
17 plt.savefig('ROC',dpi=300)
18 plt.show();
```

AUC-ROC5.py hosted with ❤ by GitHub

[view raw](#)

We use cookies on Analytics Vidhya websites to deliver our services, analyze web traffic, and improve your experience on the site. By using Analytics Vidhya, you agree to our [Privacy Policy](#) and [Terms of Use](#). [Accept](#)

AUC-ROC Curve in Machine Learning Clearly Explained



It is evident from the plot that the AUC for the Logistic Regression ROC curve is higher than that for the KNN ROC curve. Therefore, we can say that logistic regression did a better job of classifying the positive class in the dataset.

AUC-ROC for Multi-Class Classification

Like I said before, the AUC-ROC curve is only for binary classification problems. But we can extend it to multiclass classification problems by using the One vs All technique.

So, if we have three classes 0, 1, and 2, the ROC for class 0 will be generated as classifying 0 against not 0, i.e. 1 and 2. The ROC for class 1 will be generated as classifying 1 against not 1, and so on.

The ROC curve for multi-class classification models can be determined as below:

```
1 # multi-class classification
2 from sklearn.multiclass import OneVsRestClassifier
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import roc_curve
6 from sklearn.metrics import roc_auc_score
7
8 # generate 2 class dataset
9 X, y = make_classification(n_samples=1000, n_classes=3, n_features=20, n_informative=3, random_state=42)
10
11 # split into train/test sets
12 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=42)
13
14 # fit model
15 clf = OneVsRestClassifier(LogisticRegression())
```

We use cookies on Analytics Vidhya websites to deliver our services, analyze web traffic, and improve your experience on the site. By using Analytics Vidhya, you agree to our [Privacy Policy](#) and [Terms of Use](#).

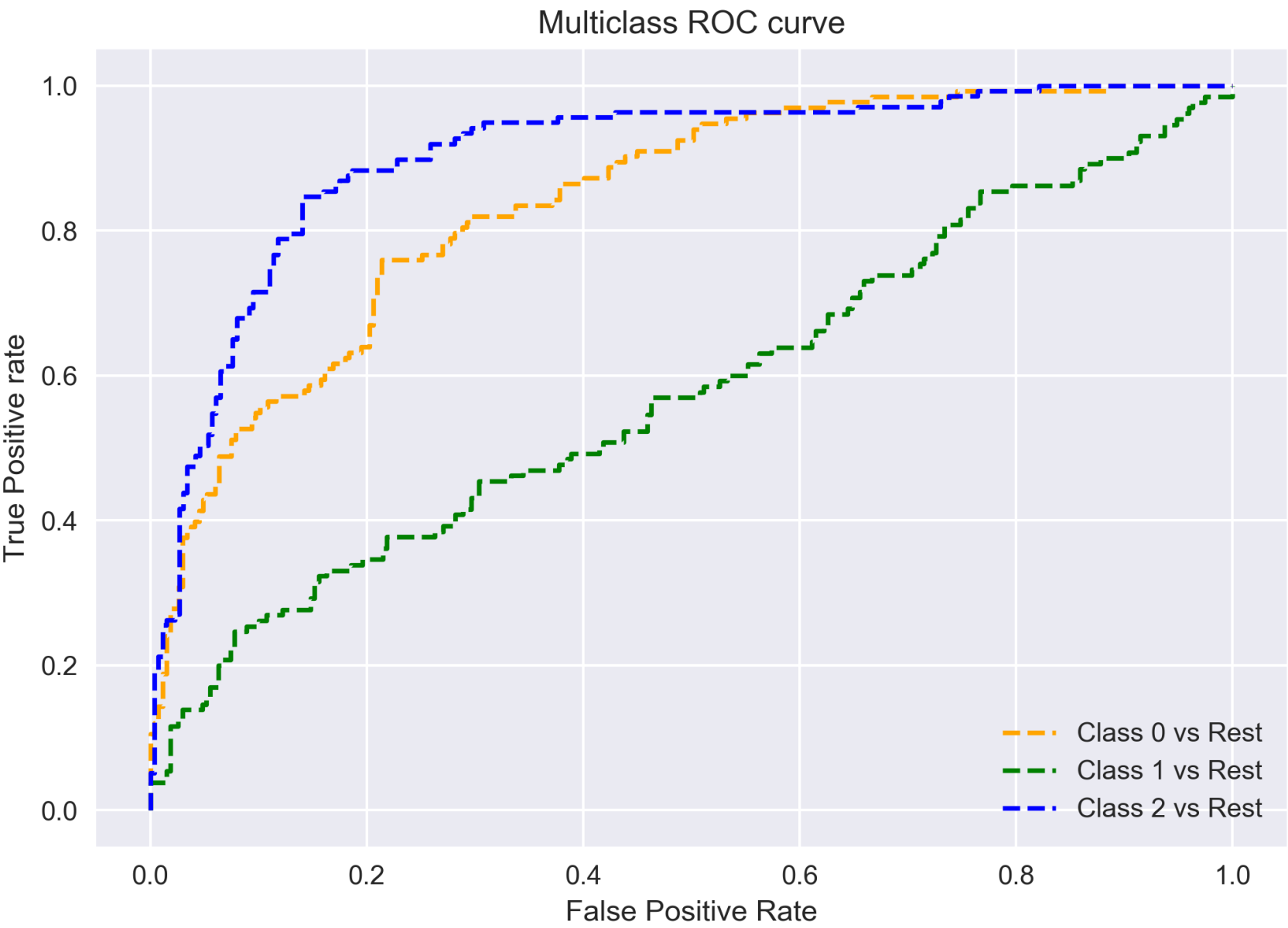


AUC-ROC Curve in Machine Learning Clearly Explained

```
21  fpr = {}
22  tpr = {}
23  thresh = {}
24
25  n_class = 3
26
27  for i in range(n_class):
28      fpr[i], tpr[i], thresh[i] = roc_curve(y_test, pred_prob[:,i], pos_label=i)
29
30  # plotting
31  plt.plot(fpr[0], tpr[0], linestyle='--',color='orange', label='Class 0 vs Rest')
32  plt.plot(fpr[1], tpr[1], linestyle='--',color='green', label='Class 1 vs Rest')
33  plt.plot(fpr[2], tpr[2], linestyle='--',color='blue', label='Class 2 vs Rest')
34  plt.title('Multiclass ROC curve')
35  plt.xlabel('False Positive Rate')
36  plt.ylabel('True Positive rate')
37  plt.legend(loc='best')
38  plt.savefig('Multiclass ROC',dpi=300);
```

AUC-ROC6.py hosted with ❤ by GitHub

view raw



End Notes

I hope you found this article useful in understanding how powerful the AUC-ROC curve metric is in measuring the performance of a classifier. You'll use this a lot in the industry and even in data science or machine learning hackathons. Better get familiar with it!

Going further I would recommend you the following courses that will be useful in building your data science career:

We use cookies on Analytics Vidhya websites to deliver our services, analyze web traffic, and improve your experience on the site. By using Analytics Vidhya, you agree to our [Privacy Policy](#) and [Terms of Use](#). Accept

AUC-ROC Curve in Machine Learning Clearly Explained



[auc roc curve](#) [confusion matrix](#) [live coding](#)

**Data Science**
Blogathon

- Total prizes worth ₹6 Lakh(≈\$8000)
- Introducing **Author Referral Program**



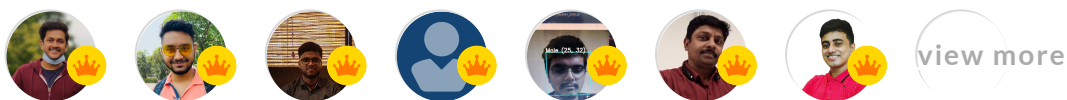
8th - 30th April 2022
[Register Now](#)

About the Author



[aniruddha](#)

Our Top Authors



Download

Analytics Vidhya App for the Latest blog/Article



Previous Post

[A Classic Computer Vision Project – How to Add an Image Behind Objects in a Video](#)

Next Post

[3 Advanced Excel Charts Every Analytics Professional Should Try.](#)

9 thoughts on "AUC-ROC Curve in Machine Learning Clearly Explained"



Ogah says:

June 16, 2020 at 3:10 pm

We use cookies on Analytics Vidhya websites to deliver our services, analyze web traffic, and improve your experience on the site. By using Analytics Vidhya, you agree to our [Privacy Policy](#) and [Terms of Use](#). [Accept](#)

AUC-ROC Curve in Machine Learning Clearly Explained



Hi Aniruddha, Thank you for writing such a comprehensive overview of AUC-ROC metrics. I just have a query regarding your example table in the Probability of Predictions section: at the bottom of the table, why is FPR=0, TNR=1 rather than the other way around? Thanks Jon

[Reply](#)



Mahtab says:
June 16, 2020 at 7:10 pm

Very grateful post for me, I want to say you tha k you so much

[Reply](#)



Yami says:
June 16, 2020 at 10:27 pm

p_fpr, p_tpr, _ = roc_curve(y_test, random_probs, pos_label=1) What to use inside random_probs, as it is throwing error

[Reply](#)



Aniruddha Bhandari says:
June 16, 2020 at 11:30 pm

Thanks, Jon for pointing it out.

[Reply](#)



Aniruddha Bhandari says:
June 16, 2020 at 11:30 pm

Glad you found it useful.

[Reply](#)



Aniruddha Bhandari says:
June 16, 2020 at 11:33 pm

It is just a list of zeros to make tpr = fpr. I have made changes to the code and you can find it in the gist named AUC-ROC3.py.

[Reply](#)



mk1332 says:
July 17, 2020 at 3:19 am

Very Nice ! The best explanations for ROC curves and AUC I have found on the internet so far

[Reply](#)



Sunkara says:
September 05, 2020 at 9:34 pm

Thanks. Very detailed explanation.

[Reply](#)

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment

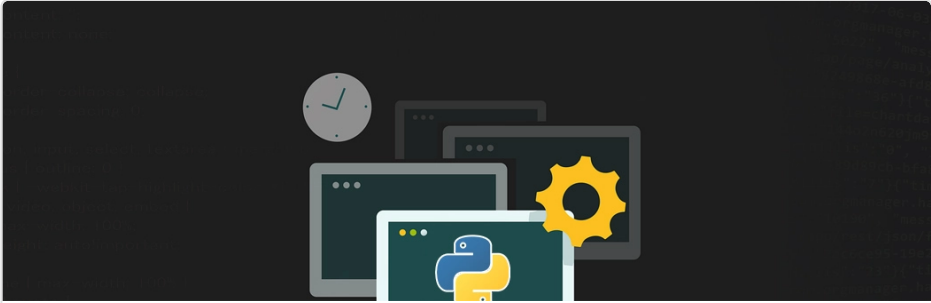
Name*

Email*

Website

We use cookies on Analytics Vidhya websites to deliver our services, analyze web traffic, and improve your experience on the site. By using Analytics Vidhya, you agree to our [Privacy Policy](#) and [Terms of Use](#). Accept

Top Resources



[Python Tutorial: Working with CSV file for Data Science](#)

 [Harika Bonthu](#) - AUG 21, 2021



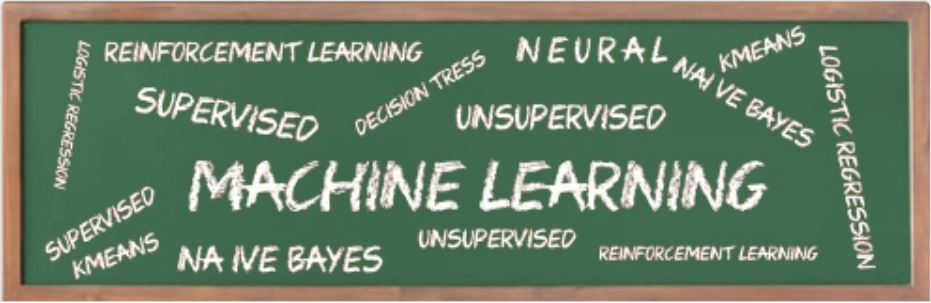
[Understanding Support Vector Machine\(SVM\) algorithm from examples \(along with code\).](#)

[sunil](#) - SEP 13, 2017



[Understanding Random Forest](#)

[Sruthi E R](#) - JUN 17, 2021



[Commonly used Machine Learning Algorithms \(with Python and R Codes\).](#)

[sunil](#) - SEP 09, 2017

Download App



Analytics Vidhya

About Us

Our Team

Careers

Contact us

Companies

Post Jobs

Trainings

Hiring Hackathons

Advertising

Data Scientists

Blog

Hackathon

Discussions

Apply Jobs

Visit us

